← Back to Portfolio

Project 5

Flow Matching and Diffusion Models

Table of Contents

Part A: The Power of Diffusion Models

Part 0: Setup and Text Prompts

For this project, I used the DeepFloyd IF diffusion model, a two-stage text-to-image model. I generated embeddings for various interesting text prompts to explore the model's capabilities. I used a consistent random seed throughout all experiments to ensure reproducibility.

Some of my creative prompts included pairs for visual anagrams and hybrid images, as well as single prompts for general image generation. The model's ability to understand and generate images from text is quite impressive, though the quality varies depending on the complexity of the prompt and the number of inference steps used.

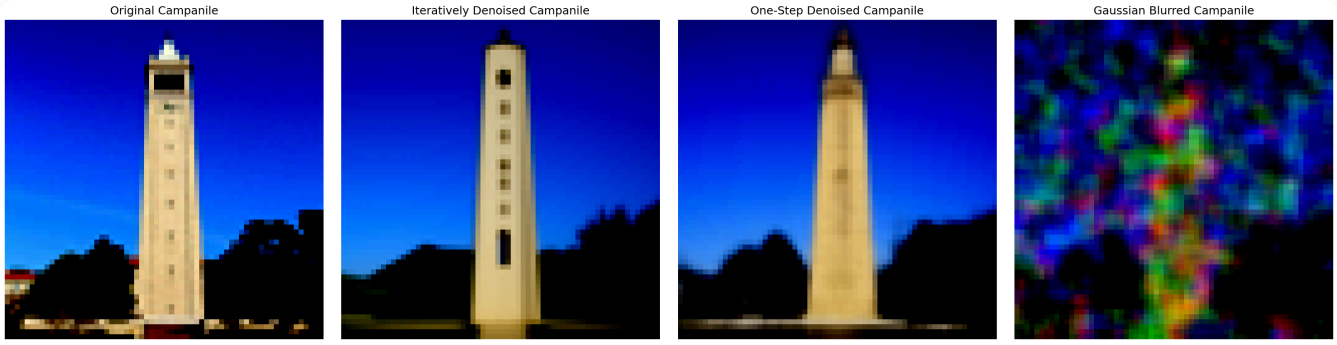
Part 1.1: Implementing the Forward Process

The forward process adds noise to a clean image progressively. I implemented this by sampling from a Gaussian distribution with mean scaled by the clean image and variance determined by the noise schedule. The formula is:

```
z_t = sqrt(ᾱ_t) * x + sqrt(1 - ᾱ_t) * ε
```

where `ᾱ_t` is the cumulative product of alphas at timestep t, x is the clean image, and ε is random Gaussian noise.

Below are the results showing the Campanile at different noise levels [250, 500, 750]. As expected, the image becomes progressively noisier:



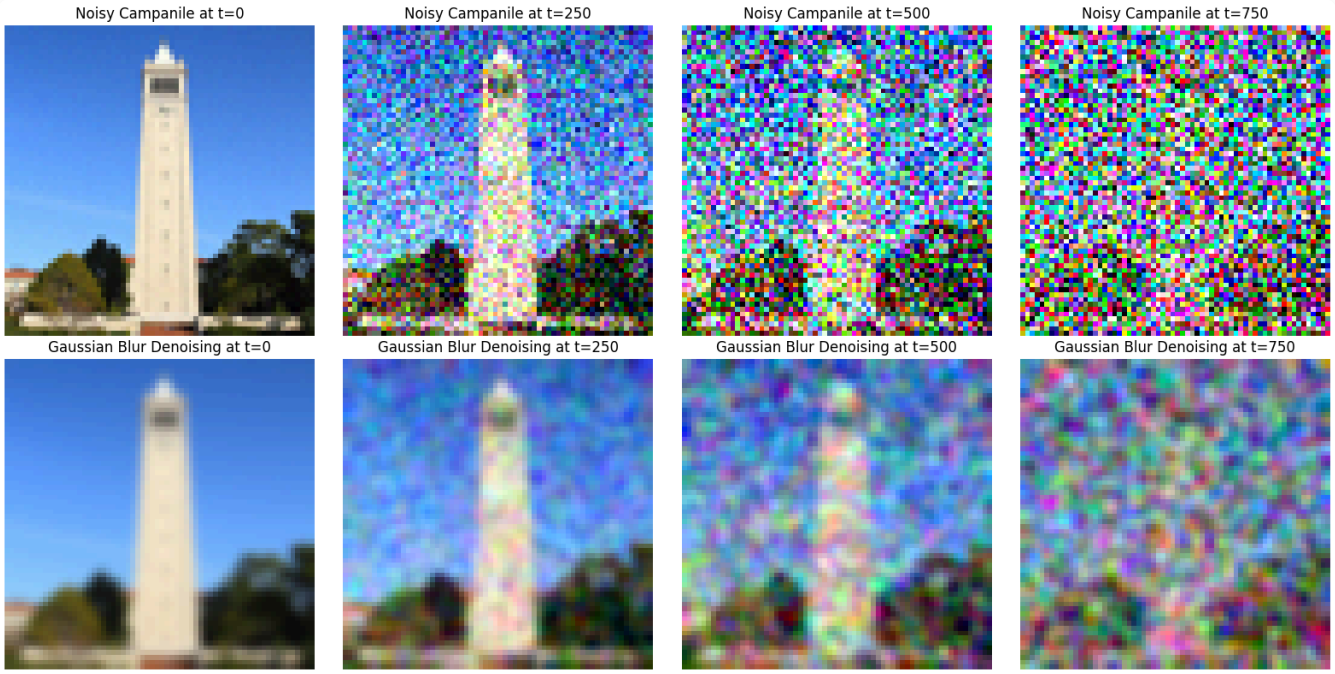
Campanile at noise levels t=250, 500, 750

Part 1.2: Classical Denoising

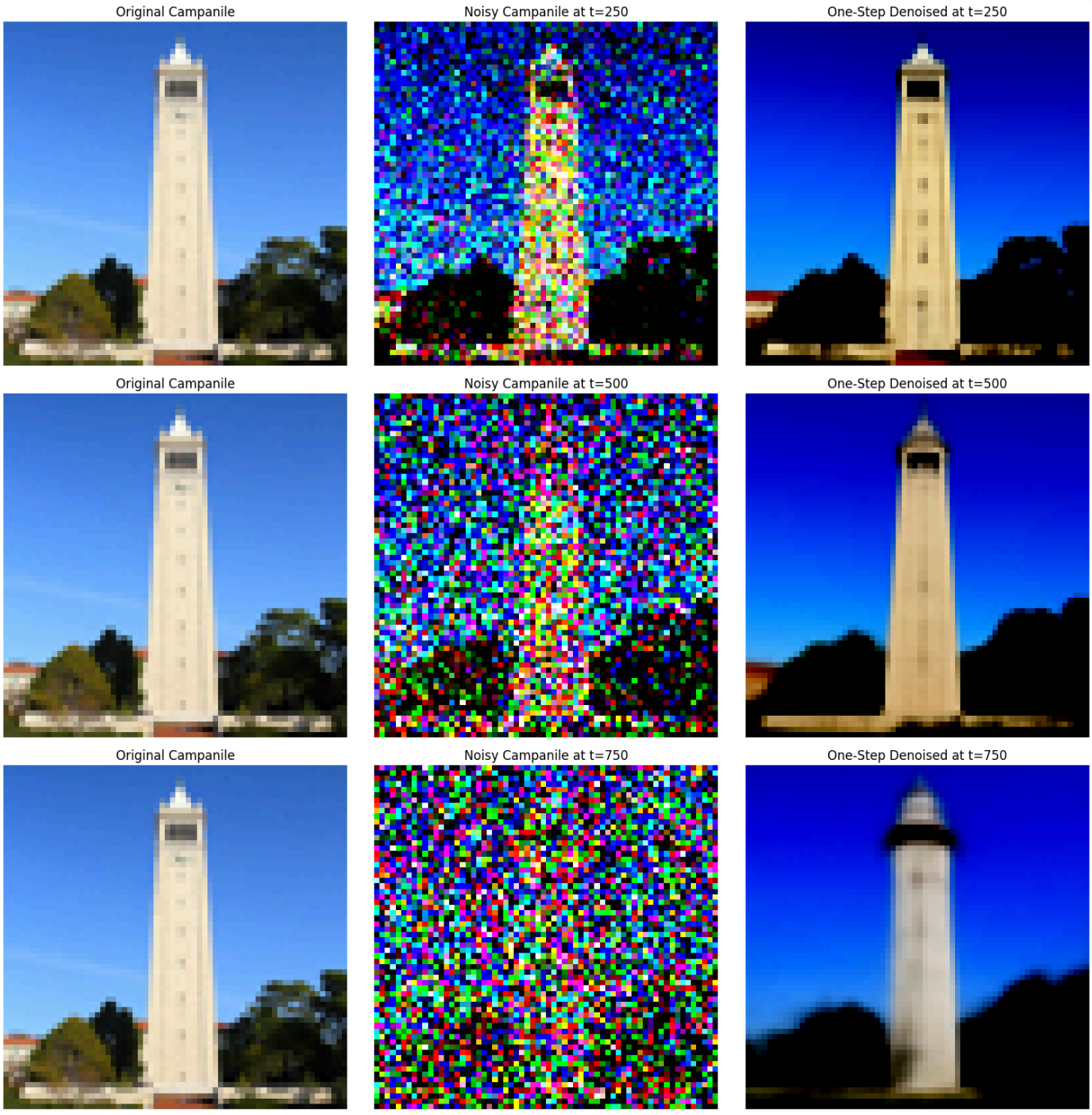
I attempted to denoise the noisy Campanile images using classical Gaussian blur filtering. This approach struggles significantly because:

- Gaussian blur cannot distinguish between noise and actual image details
- It uniformly smooths both the noise and the image structure
- Higher noise levels require stronger blur, which destroys more image detail

The results demonstrate the limitations of classical denoising methods compared to learned approaches:



Classical Gaussian Denoising (Poor Results)



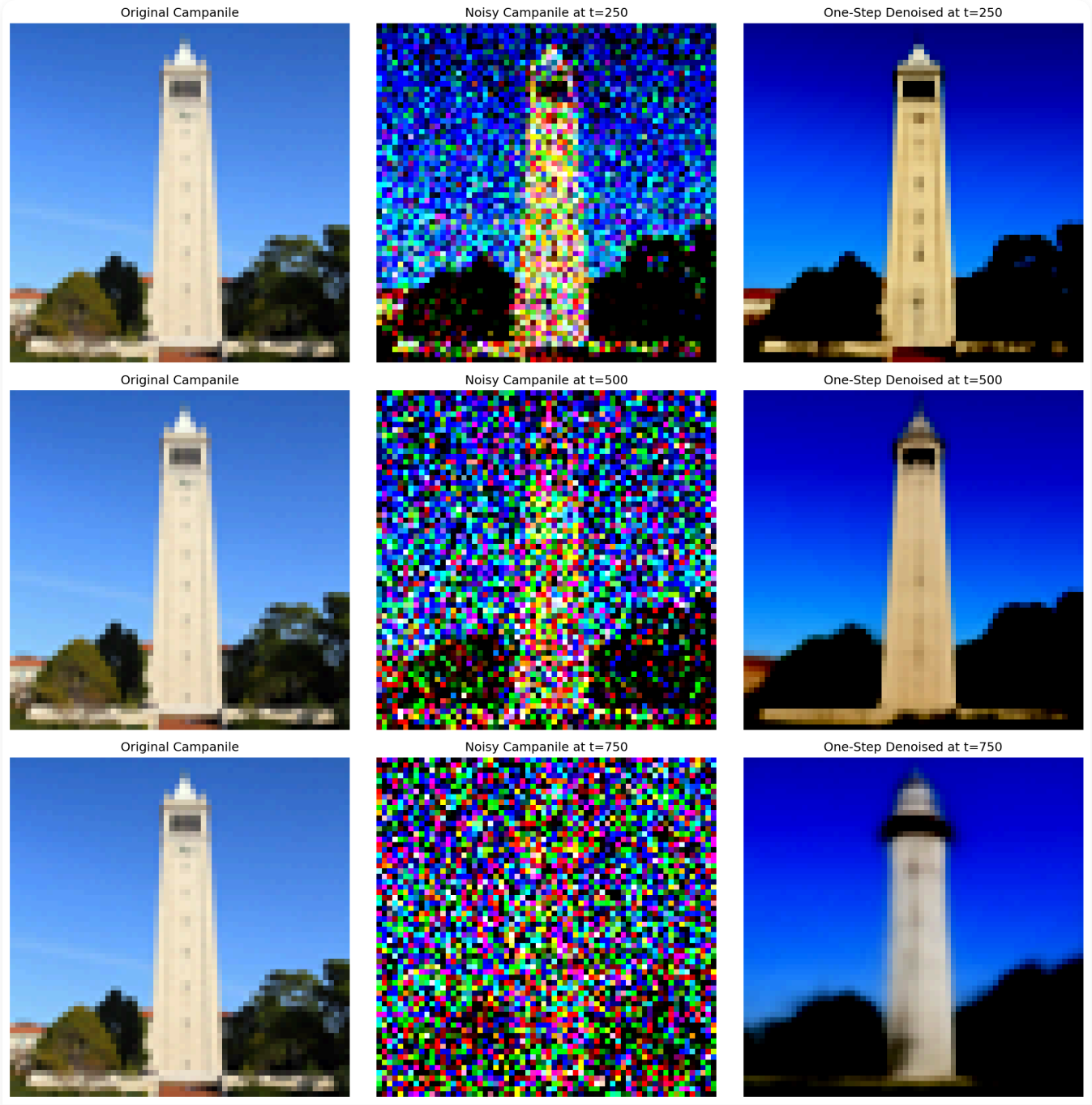
Additional Gaussian Blur Attempts

Part 1.3: One-Step Denoising

Using the pretrained DeepFloyd UNet, I performed one-step denoising by:

1. Adding noise to the Campanile image at timesteps [250, 500, 750]
2. Using the UNet to estimate the noise in the image
3. Removing the estimated noise (with proper scaling) to recover the original

The UNet does a much better job than Gaussian blur, especially at lower noise levels. However, performance degrades at higher noise levels (t=750) since the problem becomes significantly harder.



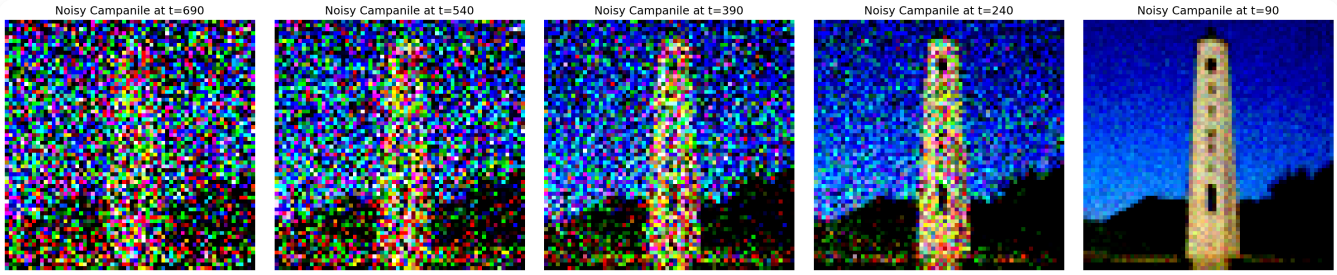
One-step denoising at t=250, 500, 750

Part 1.4: Iterative Denoising

Iterative denoising significantly improves results by denoising in small steps. I implemented this using:

- Strided timesteps: [990, 960, 930, ..., 30, 0] (stride of 30)
- Starting from a noisy image at t=300 (i_start=10)
- Progressively denoising to lower noise levels

The iterative approach works much better because each denoising step only needs to remove a small amount of noise, which is easier than removing all noise at once.



Iterative denoising progression (every 5th step shown)

Part 1.5: Diffusion Model Sampling

By starting with pure random noise and iteratively denoising, we can generate images from scratch. I generated 5 samples using the prompt "a high quality photo". The quality isn't spectacular yet (we'll improve this with CFG), but the model does produce coherent images.

5 samples generated from pure noise

Part 1.6: Classifier-Free Guidance (CFG)

Classifier-Free Guidance dramatically improves image quality by combining conditional and unconditional noise estimates:

$$\tilde{\varepsilon} = \varepsilon_u + \gamma(\varepsilon_c - \varepsilon_u)$$

where γ is the guidance scale (I used γ=7), ε_c is the conditional estimate, and ε_u is the unconditional estimate. This pushes the generation toward the conditional prompt while maintaining image quality.

5 samples with CFG (γ=7) - much higher quality!

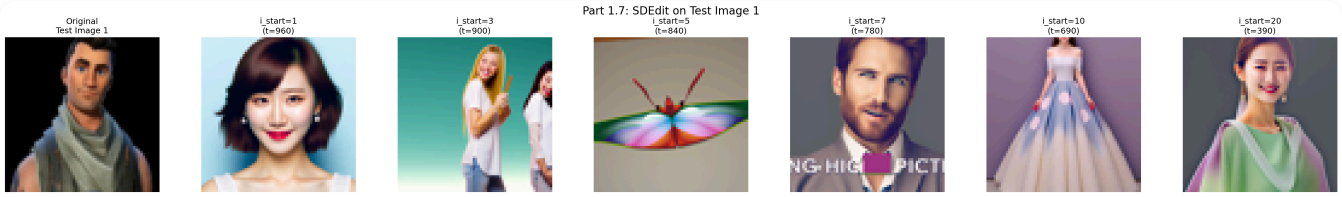
Part 1.7: Image-to-Image Translation (SDEdit)

SDEdit allows us to edit images by adding noise and then denoising. The amount of noise controls the strength of the edit - more noise allows larger changes, while less noise preserves more of the original.

I tested this on the Campanile using i_start values of [1, 3, 5, 7, 10, 20], which correspond to different amounts of initial noise. Lower i_start means more noise and more dramatic changes:

Campanile edits at different noise levels

Here are two additional test images I edited using the same procedure:



Custom test image 1 - SDEdit results

Custom test image 2 - SDEdit results

Part 1.7.1: Editing Hand-Drawn and Web Images

This technique works particularly well for projecting non-realistic images (sketches, drawings, paintings) onto the natural image manifold. I experimented with both web images and hand-

drawn sketches.

Web Images

Door image projection to natural manifold

Nite Kirk projection

Hand-Drawn Images

Hand-drawn image 1 edits

Hand-drawn image 2 edits

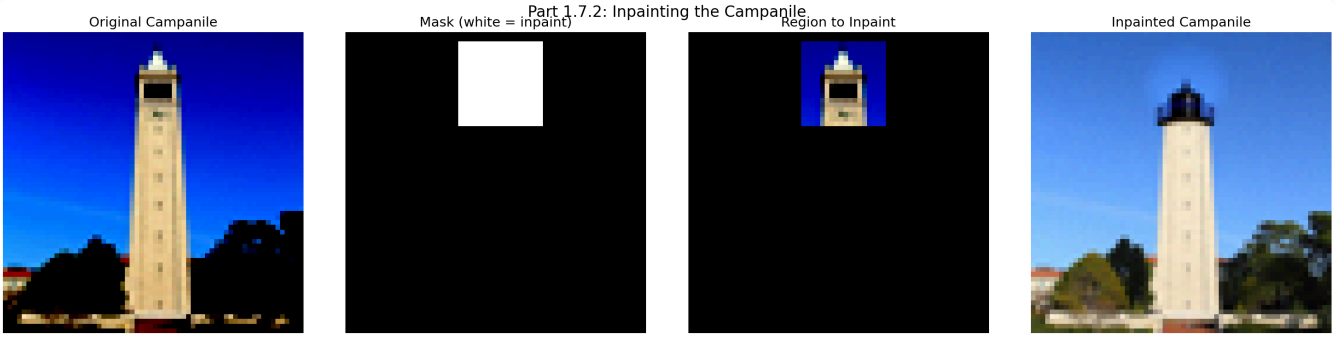
Additional hand-drawn edits

Part 1.7.2: Inpainting

Inpainting fills in masked regions while preserving unmasked areas. The algorithm works by:

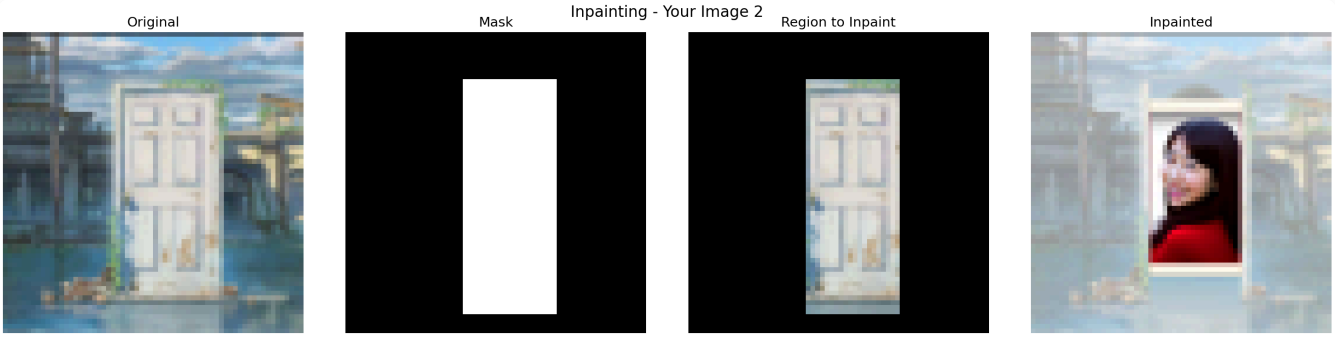
1. Running the normal denoising loop
2. After each step, replacing pixels outside the mask with the original image (with appropriate noise)
3. This forces the unmasked region to match the original while the model fills in the masked area

I implemented this using the formula: `z_t[m == 0] = forward(x_orig, t)[m == 0]`



Campanile inpainted (top section filled)

Custom image 1 inpainting result



Custom image 2 inpainting result

Part 1.7.3: Text-Conditional Image-to-Image Translation

This extends SDEdit by using specific text prompts instead of "a high quality photo". This gives us control over how the image is edited - we can guide it toward specific styles or content.

Campanile with text conditioning

Test image 1 with text prompts

Test image 2 with text prompts

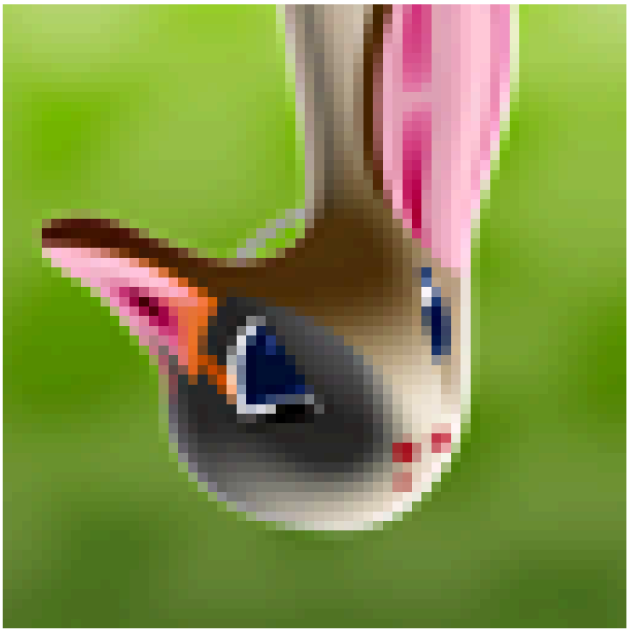
Part 1.8: Visual Anagrams

Visual anagrams are optical illusions where an image appears as one thing normally but transforms into something else when flipped upside down. I created these by:

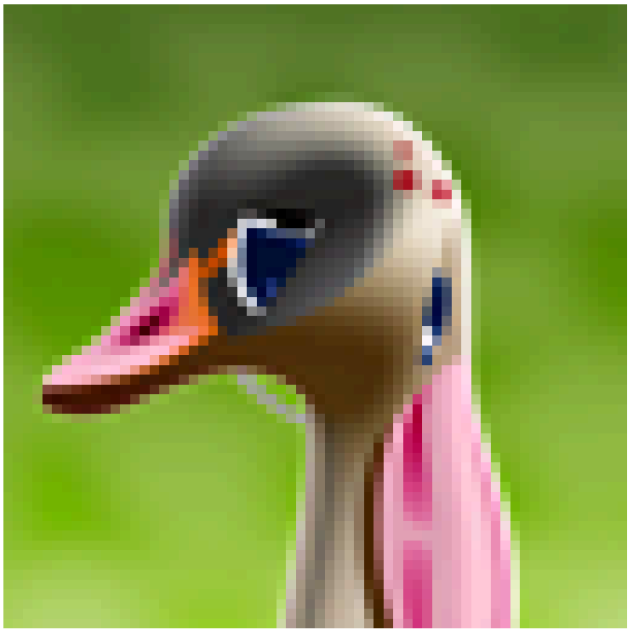
1. Computing noise estimate ε_1 for the image with prompt p_1
2. Flipping the image upside down and computing ε_2 with prompt p_2
3. Flipping ε_2 back and averaging: $\varepsilon = (\varepsilon_1 + \text{flip}(\varepsilon_2)) / 2$
4. Using this combined noise estimate for denoising

Part 1.8: Visual Anagram #1

Upright:a rabbit



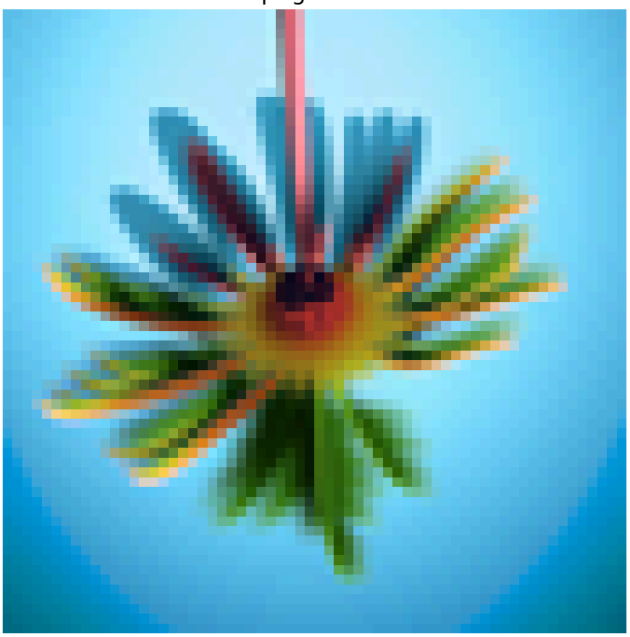
Flipped:a duck"



Visual Anagram 1 (try flipping!)

Part 1.8: Visual Anagram #2

Upright:fan"



Flipped:Palm tree"

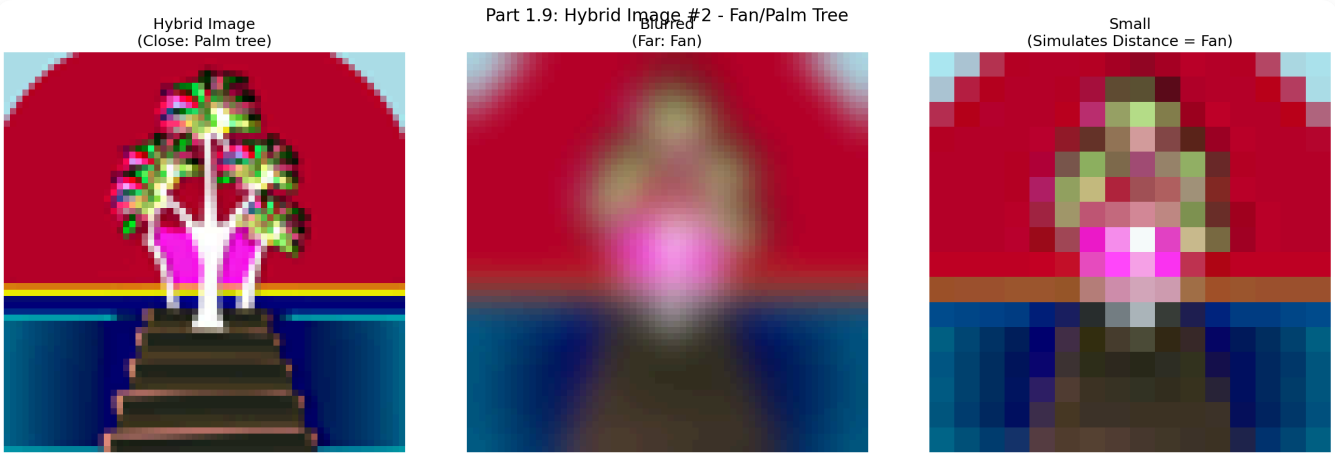
Visual Anagram 2 (flip to reveal)

Part 1.9: Hybrid Images

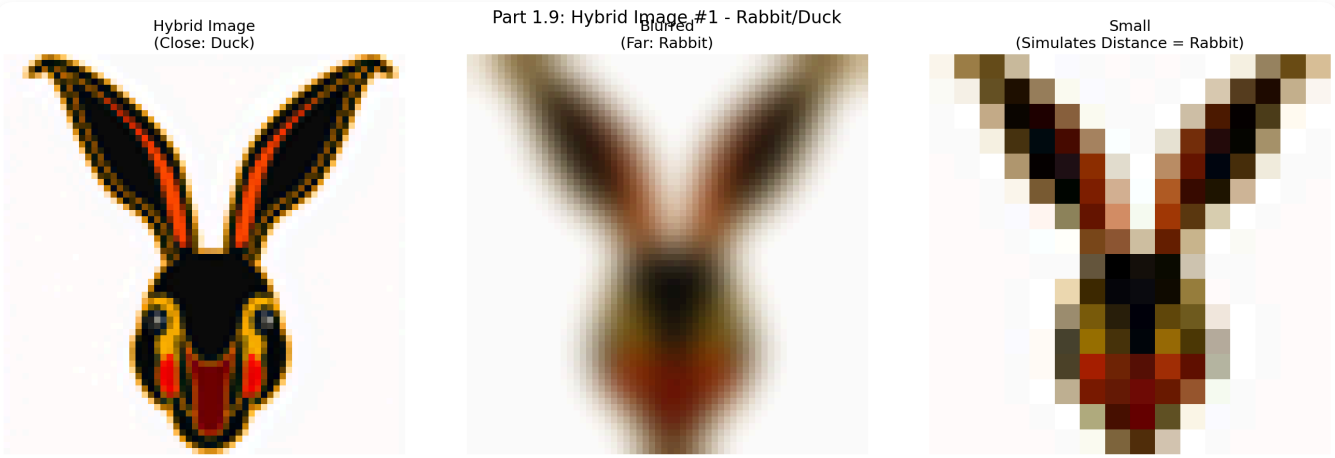
Hybrid images combine low frequencies from one image with high frequencies from another, creating images that look different from near vs. far. I implemented this by:

1. Computing noise estimates ε_1 and ε_2 for two different prompts
2. Applying low-pass filter to ε_1 and high-pass filter to ε_2
3. Combining: ε = lowpass(ε_1) + highpass(ε_2)

I used Gaussian blur with kernel size 33 and sigma 2 for the filters.



Hybrid: Fan (near) / Palm Tree (far)



Hybrid: Rabbit (near) / Duck (far)

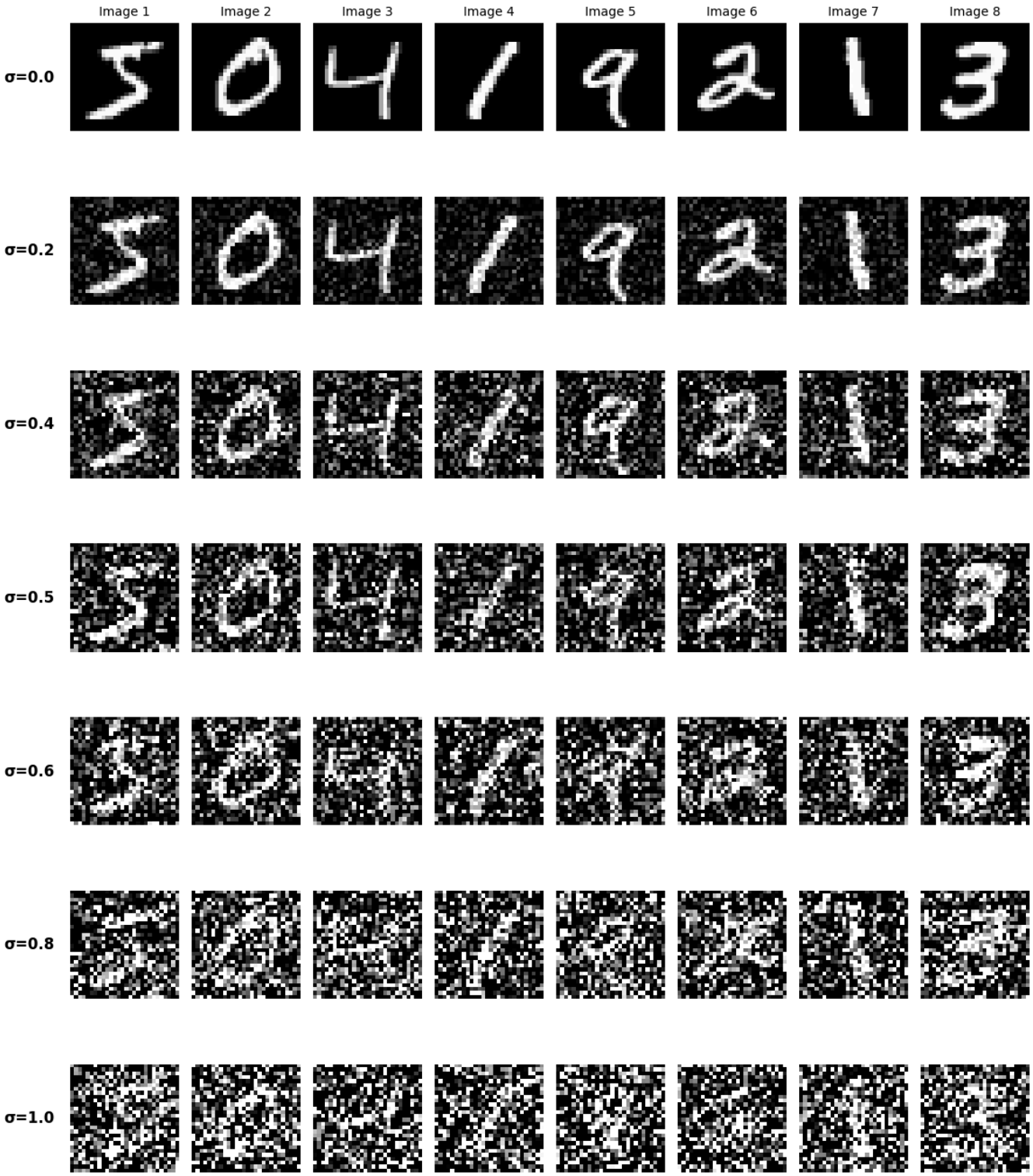
View these images from different distances to see the effect!

Part B: Flow Matching from Scratch

Part 1.2: Training a Single-Step Denoising UNet

I implemented and trained a UNet to denoise MNIST digits with noise level σ=0.5. The architecture consists of downsampling and upsampling blocks with skip connections, using:

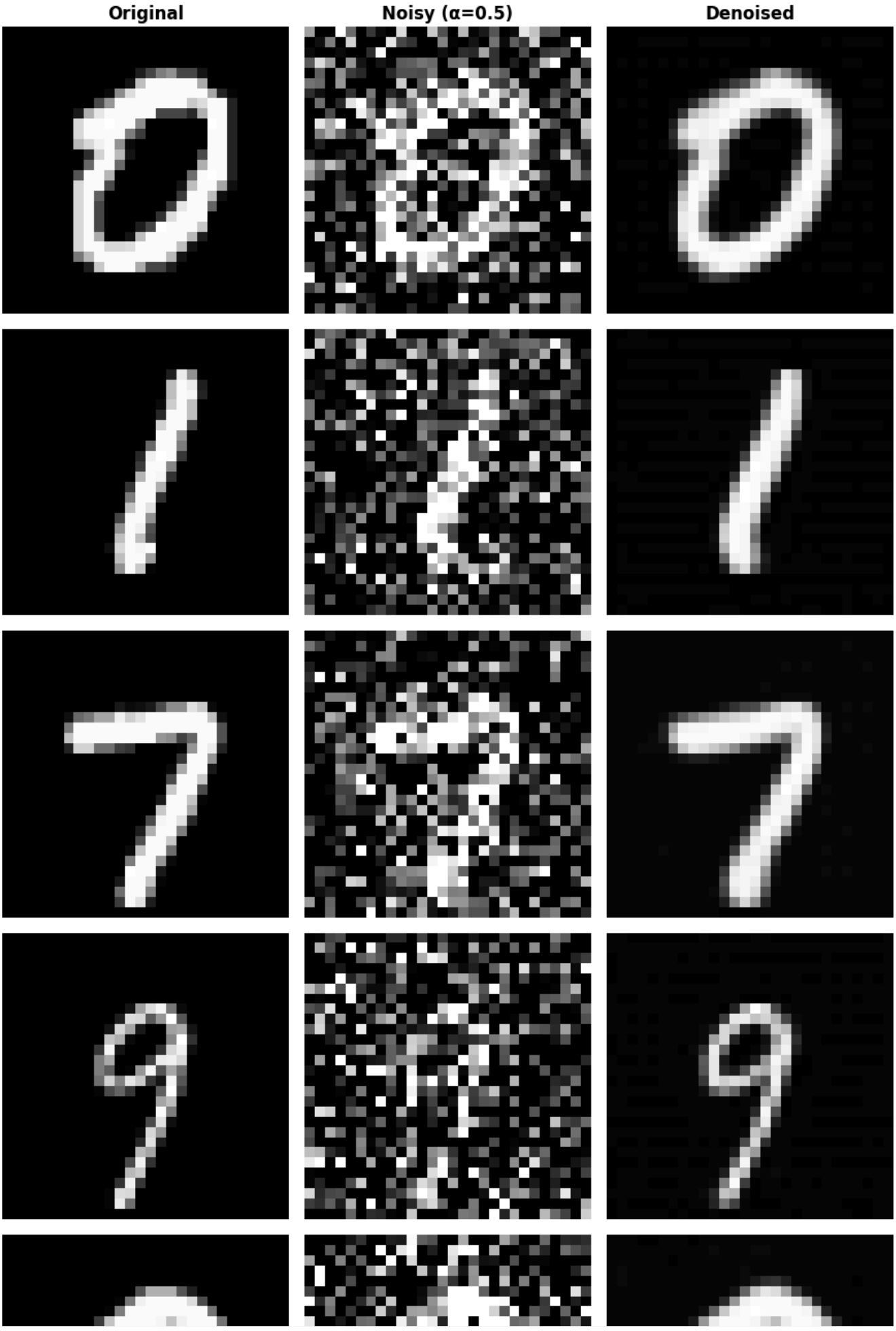
- Hidden dimension D=128
- Adam optimizer with learning rate 1e-4
- Batch size 256
- 5 epochs of training

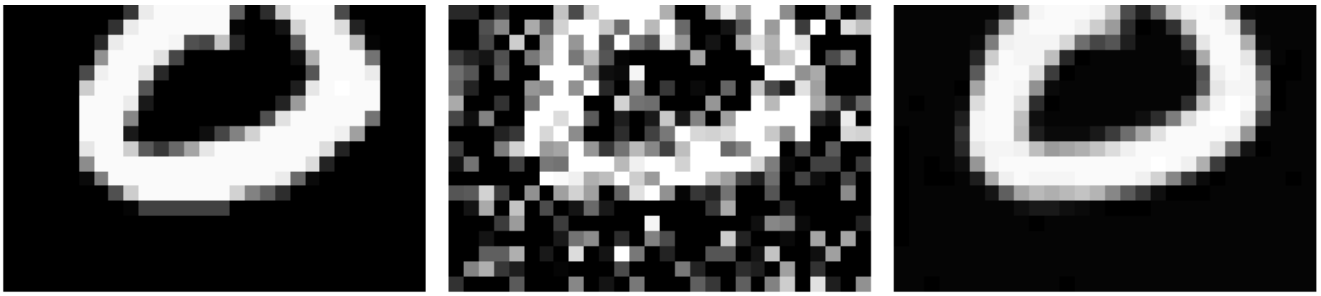


Test set images with σ=0.5 noise

Training Progress

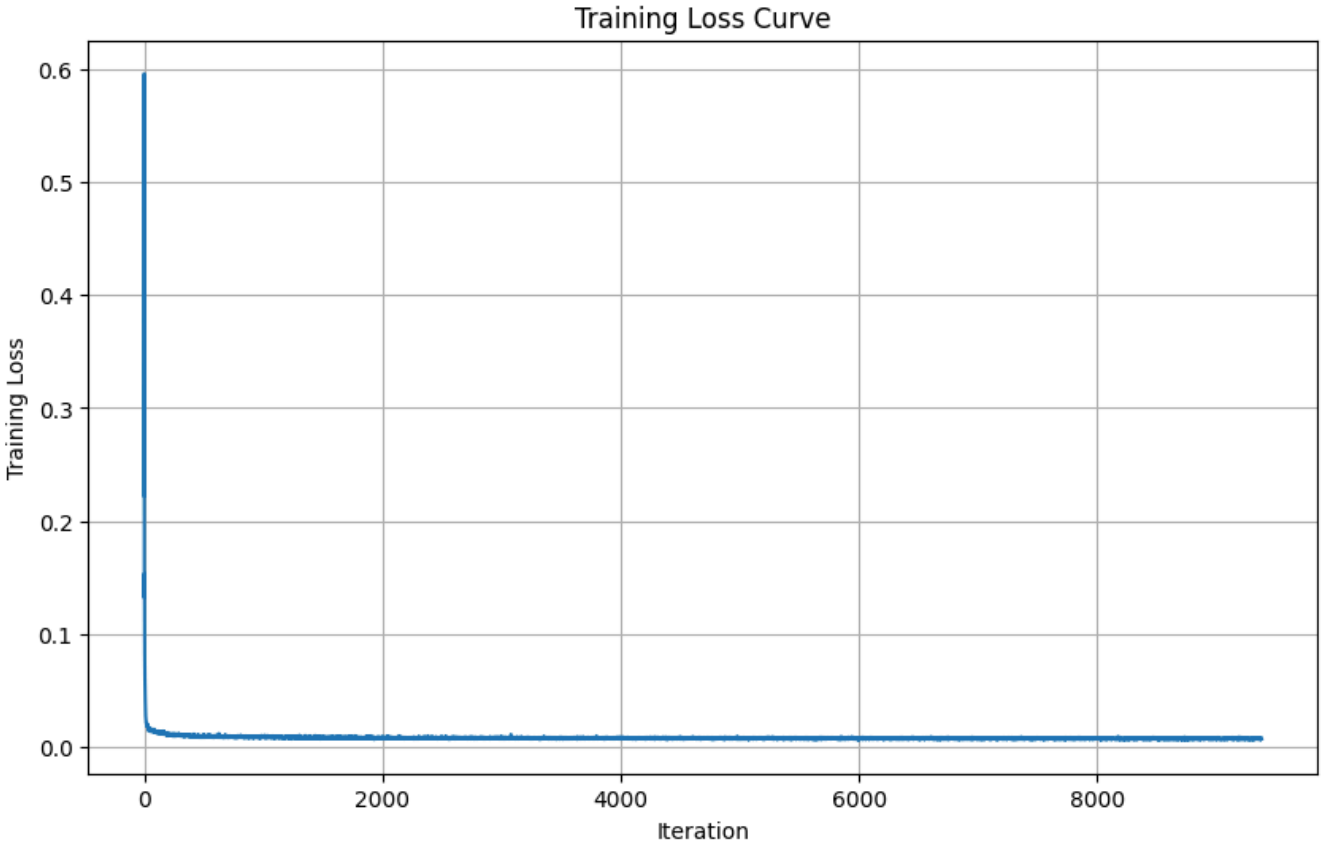
The model improves significantly from epoch 1 to epoch 5:


Original
Noisy (α=0.5)
Denoised



Denoising results after epochs 1 and 5

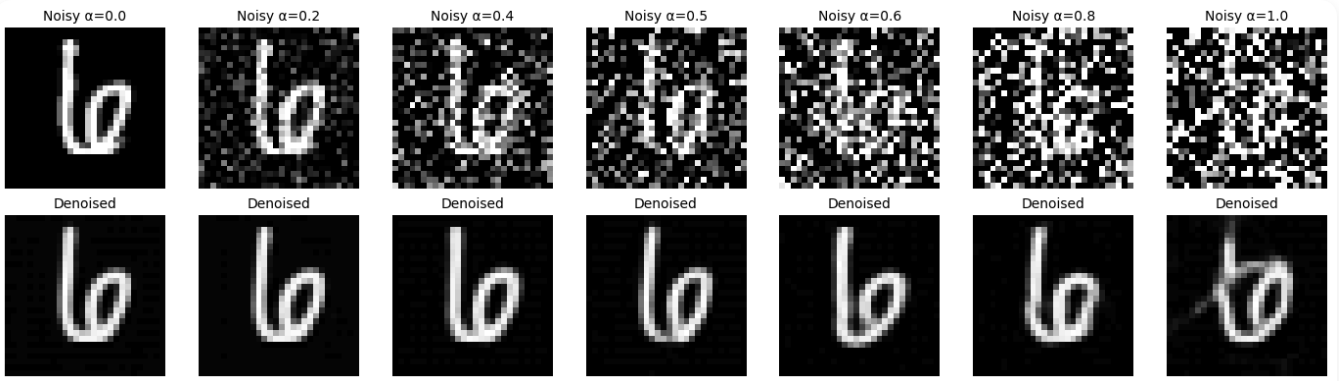
Training Loss Curve



Training loss over 5 epochs

Part 1.2.2: Out-of-Distribution Testing

I tested the denoiser on different noise levels than it was trained on. The model was trained only on σ=0.5, but I tested it on various σ values to see how well it generalizes.



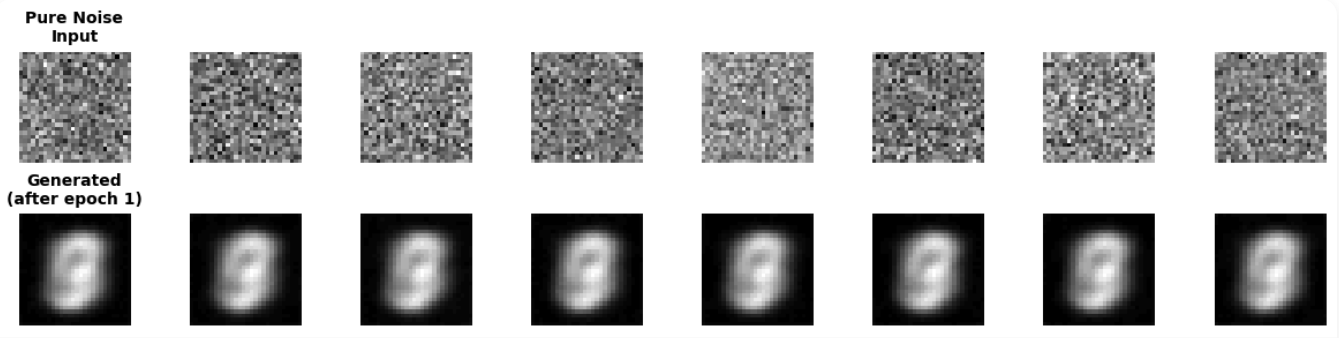
Performance on different noise levels (OOD testing)

The model performs best near σ=0.5 (its training distribution) but degrades for very different noise levels, demonstrating the importance of matching training and test distributions.

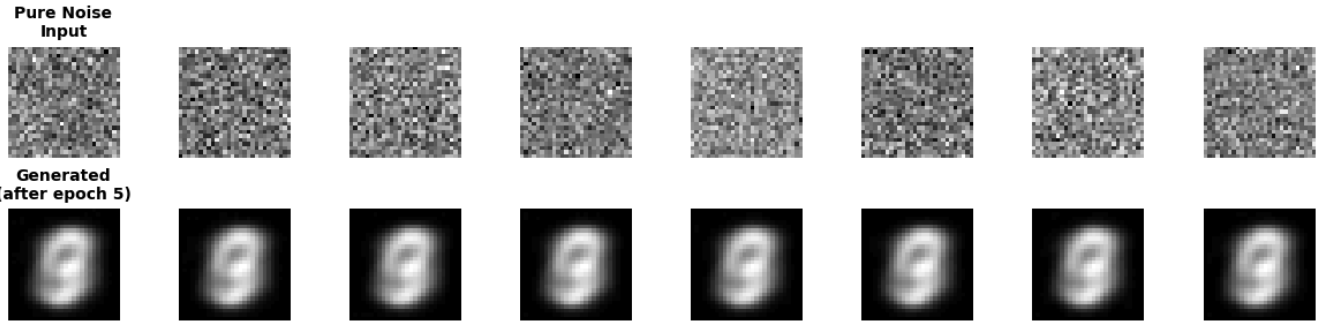
Part 1.2.3: Denoising Pure Noise

When training a denoiser to map pure noise (σ=1.0) to clean images, something interesting happens. With MSE loss, the model learns to predict the mean of all training examples - essentially a blurry "average digit".

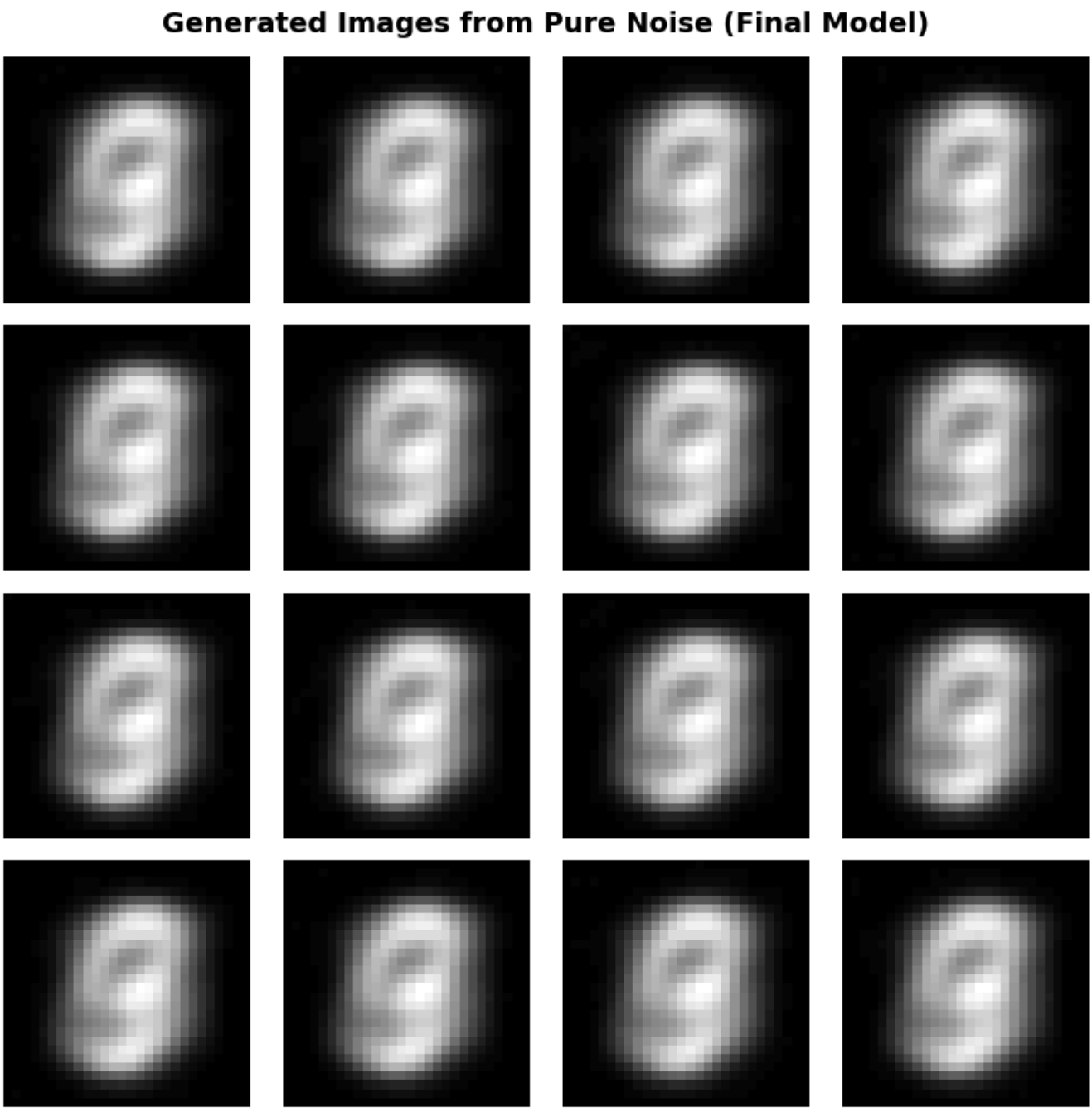
This occurs because MSE loss finds the point that minimizes squared distances to all training examples, which is the centroid. Since MNIST contains roughly equal numbers of each digit 0-9, the model converges to a blur that represents all digits simultaneously.



Pure noise denoising - Epoch 1



Pure noise denoising - Epoch 5 (converges to average)

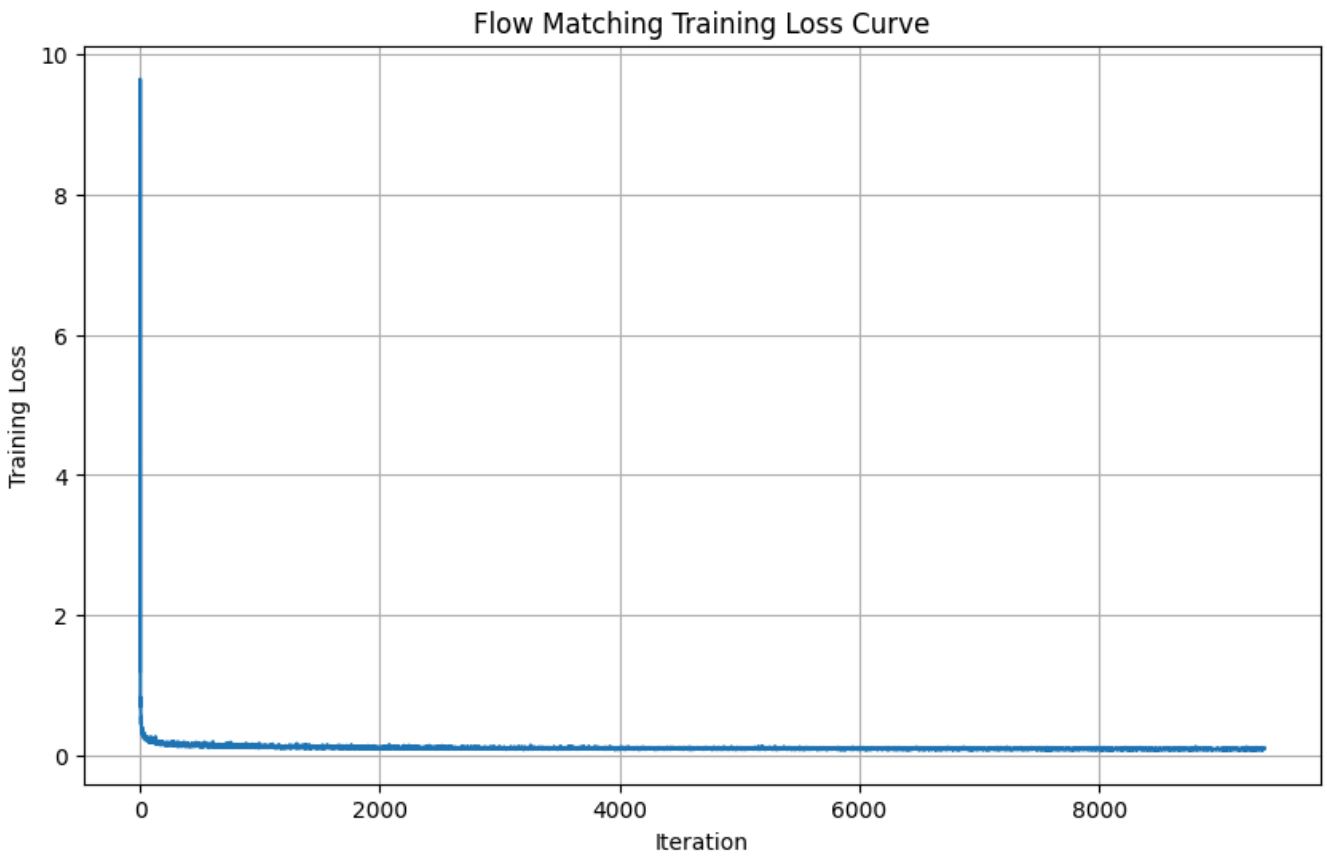


Training loss for pure noise denoising

Part 2.2: Training Time-Conditioned UNet

To enable iterative denoising, I added time conditioning to the UNet. This allows the model to know how much noise is present and adjust its denoising accordingly. Key changes:

- Added FCBlocks to inject normalized timestep t ∈ [0,1]
- Hidden dimension reduced to D=64 (faster training)
- Batch size 64, learning rate 1e-2 with exponential decay (γ=0.9999^(1/300))
- Training objective: predict flow v_t = x - z_t



Training loss for time-conditioned UNet

Part 2.3: Sampling from Time-Conditioned UNet

Using the trained time-conditioned UNet, I can now generate MNIST digits from pure noise through iterative denoising. The quality improves with more training epochs:

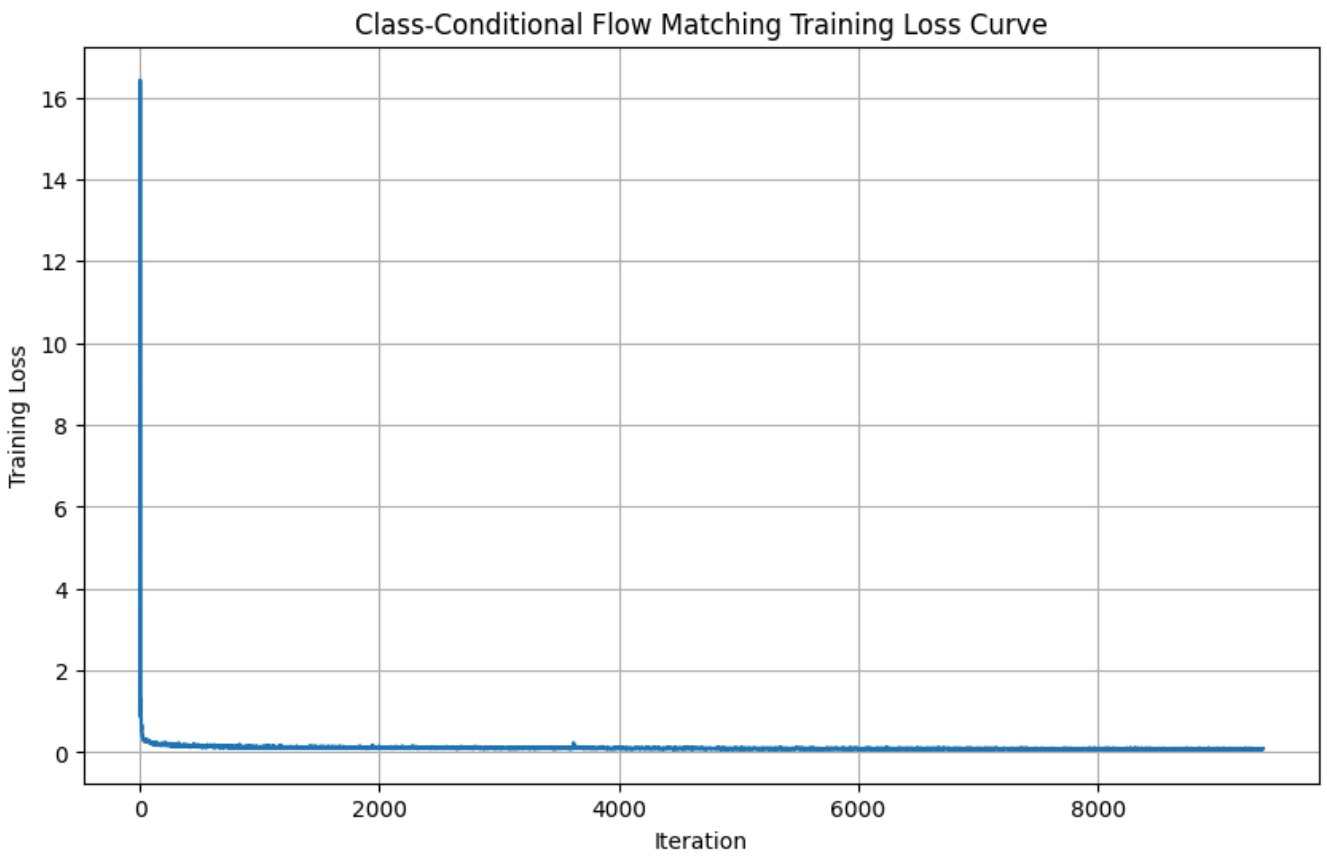
Sampling results after 1, 5, and 10 epochs

The digits are recognizable but not perfect - this is expected for unconditional generation. We'll improve this with class conditioning next.

Part 2.5: Training Class-Conditioned UNet

I added class conditioning to give us control over which digit to generate. The model takes both time t and class c as input:

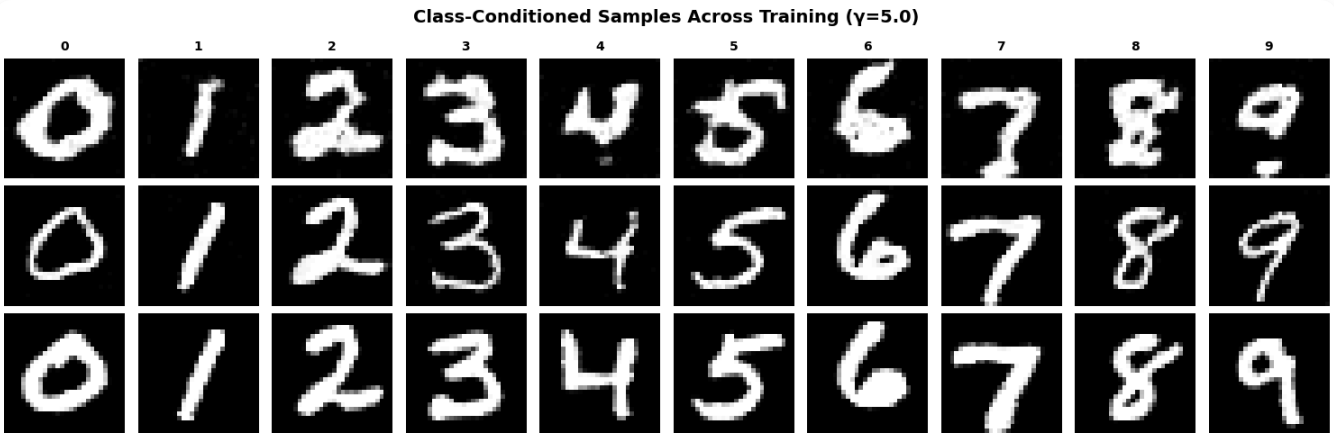
- Class is represented as a one-hot vector (10 dimensions for digits 0-9)
- 10% dropout on class conditioning (set to zero) for classifier-free guidance
- Same training setup as time-only model



Training loss for class-conditioned UNet

Part 2.6: Sampling from Class-Conditioned UNet with CFG

Using classifier-free guidance (CFG) with γ=6, I can generate high-quality digits of specific classes. The model only needs 10 epochs to converge thanks to the class conditioning:

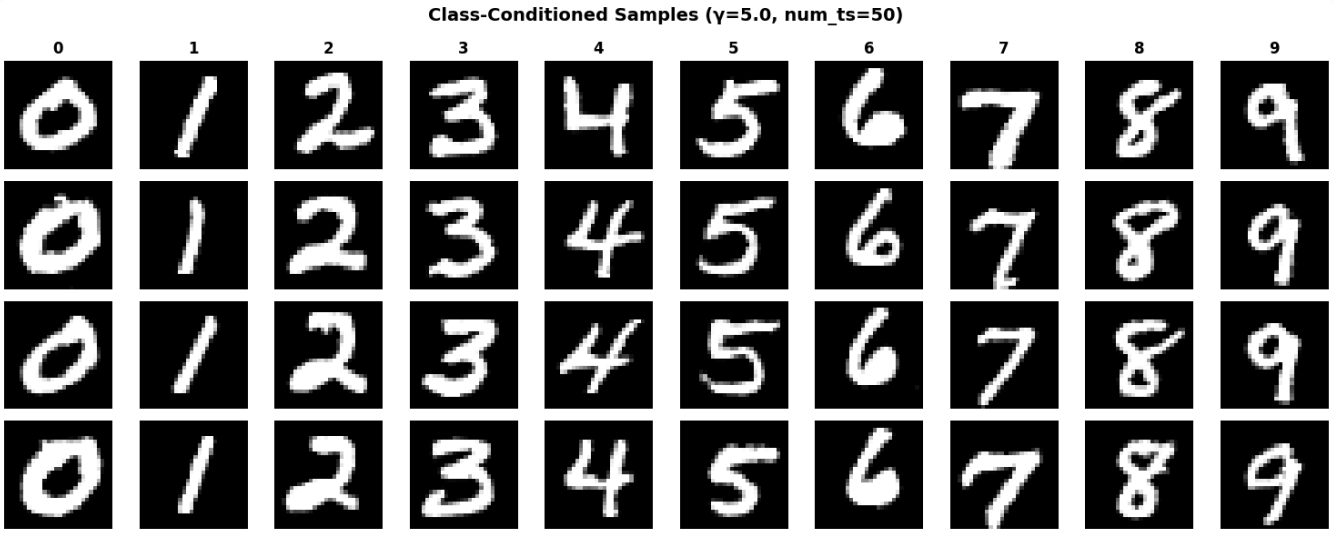


Sampling with CFG (γ=6) - 4 instances of each digit 0-9

Removing the Learning Rate Scheduler

I experimented with removing the exponential learning rate scheduler while maintaining performance. To compensate, I:

- Increased the number of training steps to 50,000 (from 10,000)
- Kept the learning rate constant at 1e-3
- This allows the model to continue learning without decay



Results without learning rate scheduler (50k steps)

The results are comparable to using the scheduler, demonstrating that with enough training steps, we can achieve similar quality without the complexity of exponential decay.